

AUBO i5 机械臂

智能分拣与视觉伺服系统

招聘展示技术文档 2025.04 – 2026.08

我们面向目标位置和摆放方向不确定的分拣场景，构建从旋转目标检测、RGB-D 几何定位到机械臂规划执行的系统，并开发基于位姿误差的视觉伺服模块。系统以眼在手外的 RealSense D435i 为主要展示方案，通过“长距离运动规划 + 近距离视觉伺服”的设计改善单次视觉定位后缺少在线修正的问题。

本文重点展示感知与抓取几何、MoveIt 执行链、SDK 硬件接口、视觉闭环算法以及线程协作机制，便于招聘交流时结合代码和演示说明工程实现。

项目指标	结果	指标含义
旋转目标检测	mAP@0.5 77.2%	检测精度，不等同于抓取成功率
目标位置与姿态估计误差	5.8 mm / 3.3°	定位与方向估计指标
平均预测耗时	5.4 ms	模型预测阶段耗时
单次运动规划耗时	< 100 ms	实验场景下的规划结果
综合抓取成功率	92.6%	完整抓取任务的结果指标

指标为项目实验记录值；模型耗时、整条感知链延迟、控制周期和任务成功率采用不同统计口径，应分别展示。详细测试口径见第八节。

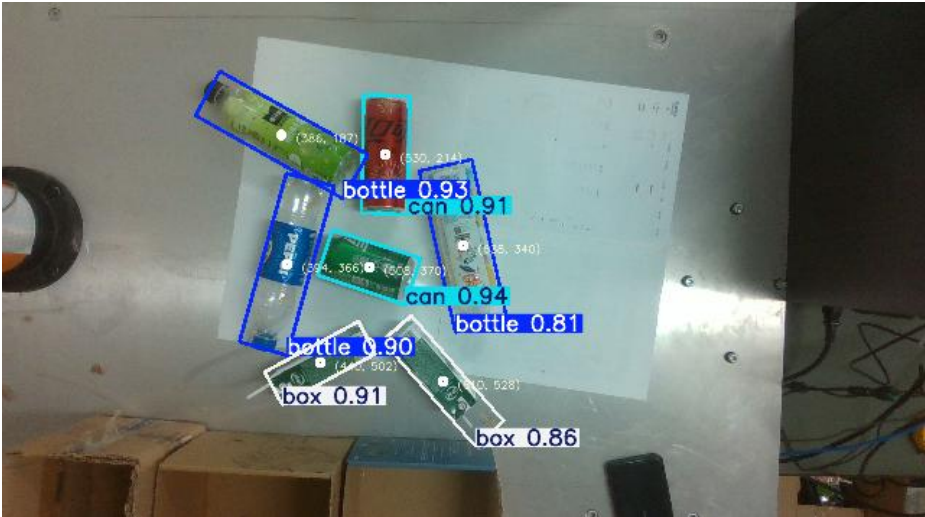
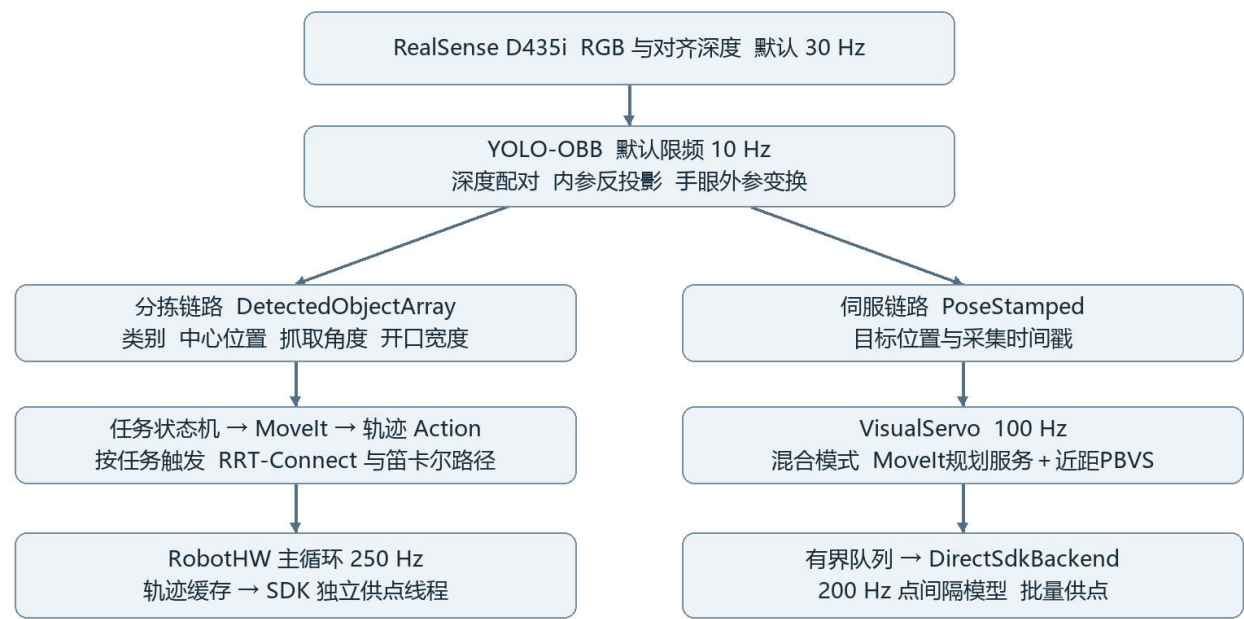


图 1 原项目文档中的旋转目标检测画面 展示瓶体 罐体与盒体的类别和方向

技术栈 ROS 1 · C++ / Python · YOLO-OBb · RGB-D · TF · MoveIt / OMPL · ros_control · AUBO SDK · KDL / Eigen

一 系统架构与模块职责

系统按感知、任务规划和机械臂执行分层。检测器负责回答“是什么、在哪里、朝向如何”；几何层将像素结果变成机器人可用的抓取参数；任务层组织抓取顺序；执行层负责轨迹或伺服位置指令与控制柜的通信。



真机两条链路互斥使用控制柜；相同 SDK 不代表相同控制进程或同一条队列。

图 2 系统运行架构 频率为当前配置或源码名义值 独立 RobotHW 与伺服 SDK 输出互斥

模块	仓库位置	核心职责
感知与几何	aubo/aubo_perception	模型接入、深度配对、坐标变换、抓取角度与宽度
分拣任务	aubo/aubo_sorting_core	目标稳定、抓放状态机、失败返回与动作执行
规划与模型	aubo/aubo_moveit_config aubo/aubo_description	URDF、关节与碰撞约束、OMPL 配置
控制与驱动	aubo/aubo_ros_control	RobotHW、视觉伺服、队列、SDK 通信
场景配置	aubo/aubo_sorting	固定机械臂工位、启动文件与抓取参数

实现边界

常规分拣通过 MoveIt Action 与 RobotHW 执行；混合控制则在同一个 VisualServo 节点中请求 MoveIt 规划服务，将远距离轨迹和近距离 PBVS 共同送入伺服队列与同一输出后端，避免两套 SDK 同时控制。完整夹取、搬运和放置仍由分拣任务层组织。[S9]

二 从旋转检测到机械臂抓取位姿

YOLO 适配节点加载外部模型工程中的 GC-yolo.pt，解析 OBB 中心、宽高、方向角、类别与置信度，发布统一检测消息。模型结构改进属于独立训练工程；ROS 侧的重点是把检测输出可靠地接入实时几何与抓取流程。[S1]

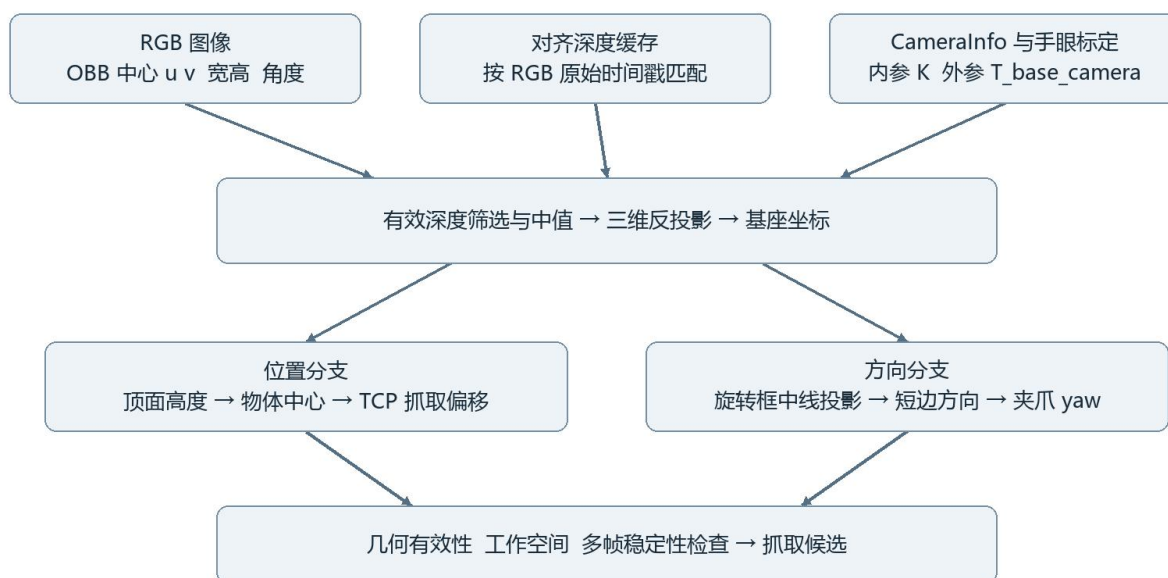


图 3 抓取几何流程 图像方向必须经过相机投影与坐标变换

深度与时间对齐

检测消息保留 RGB 采集时间戳，几何节点按该时间从深度缓存中匹配图像。默认缓存 90 帧、允许时间差 0.08 s；旋转框中心区域筛除无效及越界深度后取中值，有效样本少于 5 个则拒绝该目标。这样可减小孔洞、背景和推理延迟引入的定位偏差。[S2]

反投影与手眼变换

对中心像素(u, v)与深度 Z，计算 $X = (u - cx)Z / fx$ 、 $Y = (v - cy)Z / fy$ ，再由 $p_base = R_base_camera p_camera + t_base_camera$ 转换至机械臂基座系。内参来自 CameraInfo；眼在手外的外参由标定结果发布，安装位置变化后必须重新标定。

位置和方向的工程处理

深度读数对应物体可见顶面。代码先校验顶面与桌高加物高是否一致，再减去半个物高得到物体中心，最后叠加 TCP 抓取偏移。另有 table 模式，用相机射线与已知水平面求交；该模式依赖桌高和物高先验，不能与深度定位混为一谈。

方向计算将旋转框两条中线投影到基座水平面，选取米制短边作为夹爪闭合方向，再叠加夹爪轴标定偏置。该方法表达桌面抓取位置和偏航角，并非任意物体完整 6D 姿态重建。抓取宽度控制还需要夹爪开口标定，默认不启用。

三 MoveIt 规划与 SDK 执行链

抓放任务如何组织

任务先完成初始化与观察位运动，再积累目标检测样本并检查稳定性。当前目标缓存按类别组织，每类维护一个候选；它适合当前分拣工位，不应描述为同类多目标身份跟踪。通过检查的目标进入预抓取、接近、闭合、抬升、搬运、放置和退离流程。[S3]

阶段	执行方式	关键检查
预抓取与搬运	MoveIt 位姿规划	关节限制、场景碰撞、规划与执行返回值
接近与放置	笛卡尔路径及时间参数化	路径完成比例；必要时按代码分支重试或回退
抬升	抬升路径与恢复逻辑	不把未完成的部分抬升当成达到安全高度
夹取与释放	夹爪 FollowJointTrajectory	动作超时、位置容差与抓取几何合法性

运动规划设置

URDF 与 SRDF 提供机器人运动链、碰撞模型和规划组；关节范围、速度与加速度缩放约束运动。OMPL 配置中默认规划器为 RRTConnect；长距离运动由其求解，局部直线动作通过 computeCartesianPath 实现。规划场景中的桌面和障碍物参与碰撞检查，点云与 OctoMap 能力可按场景接入。[S4]

配置中的 planning_time 为 12 s，表示允许的规划时间预算。“单次规划<100 ms”是实验结果，两者并不冲突，也不能用 12 s 配置或单次快速成功来证明所有工况都小于 100 ms。

ros_control 硬件接口

我们基于 AUBO SDK 封装 AuboHardwareInterface，继承 RobotHW 并注册关节状态接口与位置命令接口。MoveIt 将带时间参数的轨迹交给 FollowJointTrajectory 动作接口，JointTrajectoryController 在 controller_manager 更新中生成各关节位置命令。[S5]

主线程以默认 250 Hz 执行 read → update → write。SDK 状态定时器以 50 Hz 更新反馈缓存，read 读取该缓存；write 进行指令合法性与步长检查，并将需要执行的位置点推入单生产者单消费者队列。独立供点线程查询控制柜缓冲余量，批量取点、执行约束检查后下发。

线程解耦的作用

轨迹计算、关节状态读取和控制柜供点分开调度，降低网络查询阻塞控制器更新的影响。批量写入减少通信调用次数，但供点轮询频率不等于控制柜执行采样率。该路径的限幅常量 CTRL_PERIOD_S 为 5 ms，与 250 Hz 主循环的 4 ms 周期不同，修改频率前应同时核对点间隔与控制柜实际消费节拍。

四 视觉伺服算法与前向位置队列

视觉伺服采用基于三维位姿误差的 PBVS。眼在手外时，期望 TCP 位置等于目标基座坐标加抓取偏移；当前 TCP 位置和雅可比由关节反馈及 KDL 运动学链计算。控制器输出关节速度，再积分为 SDK 接受的关节位置点。[S6]

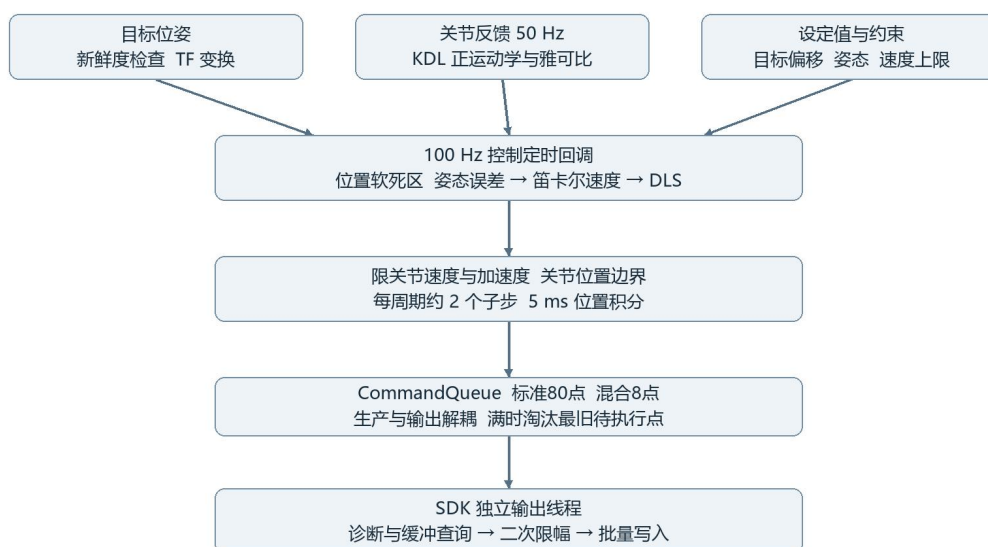


图 4 伺服计算与 SDK 执行 控制回调 100 Hz 名义位置点间隔 5 ms

控制律

位置误差 $e_p = p_{\text{target}} + p_{\text{offset}} - p_{\text{TCP}}$ 。逐轴应用连续软死区后， $v = K_p e_p$ ；姿态误差取旋转矩阵对数， $\omega = K_R \text{Log}(R_{\text{desired}} R_{\text{TCP}}^T)$ 。笛卡尔速度先限幅，再通过阻尼最小二乘得到 $\dot{q} = J^T(JJ^T + \lambda^2 I)^{-1}[v; \omega]$ 。阻尼项减轻近奇异位形下逆解的数值放大。

关节速度经过速度与加速度限幅，以 $q_{\text{next}} = q_{\text{cmd}} + \dot{q} \Delta t$ 积分并限制在关节位置边界内。100 Hz 控制回调按实际间隔生成子步，默认每 10 ms 约生成两个 5 ms 位置点；少量反馈融合抑制积分位置与实际关节位置长期偏离。这里是运动学约束与位置积分，不是完整动力学或力矩控制。

队列与姿态含义

标准伺服队列容量为 80 点，混合模式配置缩短为 8 点；满时淘汰最旧待执行点；状态切换时清除失效命令，输出端再次限速和限加速度。眼在手外当前姿态控制跟踪配置的固定抓取姿态，YOLO 伺服目标的位置接口没有直接承载 OBB 抓取 yaw；分拣链路中的角度估计与伺服姿态控制应分别说明。

五 线程回调与执行频率

下表列出源码和默认配置中的运行节拍，不代表实机测得的稳定频率。ROS 定时器由回调线程池执行；每个定时器并不对应一个独立线程。SDK 供点循环包含通信时间和休眠，因此“4 ms 休眠”只能表示约 250 Hz 的轮询上限。[S5–S8]

执行单元及职责	调度方式	默认频率或周期	作用
RealSense RGB 与深度	驱动采集	30 Hz / 33.3 ms	真机启动沿用相机驱动默认值
固定相机仿真	Gazebo 传感器	20 Hz / 50 ms	workspace_camera 的 update_rate
YOLO 推理	图像回调 + 限频锁	最高 10 Hz / ≥ 100 ms	单调时钟限频；忙时跳过
RGB-D 几何计算	检测消息回调	随有效检测到达	不是独立固定频率循环
目标稳定等待	任务线程内部轮询	10 Hz / 100 ms	waitForObject 等等待逻辑
任务初始化与抓放	独立工作线程	按任务触发	规划和动作等待，不固定 Hz
任务 ROS 回调	AsyncSpinner(4)	4 个工作线程	线程数不代表 4 Hz
RobotHW 主控制	主线程循环	250 Hz / 4 ms	read → update → write
RobotHW 状态读取	ROS 定时回调	50 Hz / 20 ms	仅状态模式降低为 1 Hz
RobotHW SDK 供点	独立线程	每轮末休眠 4 ms	缓冲不足时批量补点
硬件节点 ROS 回调	AsyncSpinner(2)	2 个工作线程	状态定时器和回调服务

线程与频率的区别

感知与任务节点按消息或请求工作，不能为每个函数指定固定 Hz。异步线程池允许状态读取、停止请求和任务回调并发处理；独立 SDK 线程负责网络诊断与供点。规划耗时与动作执行时间由任务复杂度决定。

五 视觉伺服线程与时序

视觉伺服在一个节点中组合控制定时器、反馈定时器、输出后端和异步回调线程池。混合模式另外启用规划检查定时器，规划服务调用期间释放控制锁，避免阻塞其余控制回调。

执行单元及职责	调度方式	默认频率或周期	作用
视觉伺服控制	ROS 定时回调	100 Hz / 10 ms	误差、逆解与子步积分
视觉伺服状态读取	SDK 模式定时回调	50 Hz / 20 ms	读取并发布关节反馈
Gazebo 伺服输出	ROS 定时回调	200 Hz / 5 ms	消费队列并发布关节位置
SDK 伺服输出	独立线程	每轮末休眠 4 ms	位置点按 200 Hz 模型限幅
混合规划检查	ROS 定时回调	10 Hz / 100 ms	有请求才调用 MoveIt 服务
伺服 ROS 回调及监测	AsyncSpinner(3) + 主线程	3 个回调线程；监测 10 Hz	监测不是控制计算循环

100 Hz 控制与 200 Hz 位置点

一个名义 10 ms 控制周期内，先以目标误差求一次关节速度，再分两个约 5 ms 子步完成约束与位置积分。Gazebo 输出按 200 Hz 消费；SDK 线程按缓冲需求批量写点，每轮末尾休眠 4 ms，并不意味着每轮只执行一个位置点。

标准模式的 80 点软件队列，在 200 点/秒的名义节拍下对应最多约 0.4 s 的点数规模；混合模式缩短为 8 点，约 0.04 s。这只是队列容量对应的时间，不是实际延迟；总延迟还取决于控制柜缓存与网络。

控制器附属发布频率

controllers.yaml 中 JointStateController 发布频率为 50 Hz；轨迹控制器状态发布为 25 Hz、Action 监测为 20 Hz。它们分别用于状态输出和动作状态监测，不改变 RobotHW 主循环的默认 250 Hz。

视觉频率与控制频率如何配合

默认 10 Hz 视觉观测之间，100 Hz 控制器可能使用同一帧新鲜目标多次计算。5.4 ms 平均模型预测耗时不等于 185 Hz 整机视觉闭环；实际链路还包含采集、排队、深度配对、TF、控制回调和 SDK 缓冲延迟。

六 混合控制流程与状态切换

混合控制在同一个 VisualServo 节点中完成远距离规划与近距离 PBVS 切换。通过 `/plan_kinematic_path` 只请求规划结果，校验关节顺序、时间、限位及起终点后，以统一队列输出轨迹采样点；该路径不使用 MoveIt 轨迹 Action 执行。[S9]



目标超时、规划失败或 SDK 故障时，转入保持或失败处理，不继续抓取。

图 5 同一输出后端下的混合控制流程与后续抓放接口

切换条件与规划并发

`hybrid_enabled` 默认关闭。启用后，TCP 到期望 TCP 的距离 ≤ 0.10 m 进入 PBVS， > 0.16 m 退出近距状态，迟滞避免边界反复切换。远距路径规划到距期望位 0.06 m 的接近点，预算 3 s，执行超时 30 s。规划回调按 10 Hz 检查待处理请求，服务调用期间不持有控制锁，其余回调仍能处理停止和反馈。

对齐与目标丢失

混合路径完成后先稳定 0.3 s，再进入近距控制；目标位移超过 0.04 m 会使旧路径失效。当前默认位置死区为 4 mm、姿态死区为 0.02 rad，持续满足对齐条件 0.35 s 后进入 ALIGNED 并保持反馈位置；误差超过释放阈值后恢复跟踪。目标超时阈值为 0.20 s，眼在手外默认采用 stop 策略；混合模式目标丢失时清除旧路径并保持。上述阈值是控制参数，不能视为实测终端抓取精度。

固定相机默认 `target_offset` 为 `[0, -0.08, 0.10]` m，包含避遮挡横向偏移和 10 cm 预抓取高度。因此 ALIGNED 表示到达设置的对准位，并不表示夹爪已经接触物体；最终接近动作应单独组织。

七 工程难点与实现取舍

问题	处理方式	需要解释的边界
深度噪声与推理滞后	保留采集时间；有界深度缓存；ROI 中值	拒绝无效结果，避免用旧位姿伪装新观测
图像角度与夹爪方向不同	旋转框中线投影到工作平面	依赖水平面和物体几何先验
指令生产与 SDK 通信不同步	生产消费解耦；按控制柜余量批量供点	队列容量和轮询 Hz 不是闭环带宽
近目标抖动与逆解放大	连续软死区；DLS；关节限速限加速度	不能替代规划场景中的在线避碰
目标丢失与重新捕获	超时状态机；保持；按模式选择恢复策略	眼在手外默认不执行搜索运动
规划与伺服争用控制柜	混合模式共用队列和唯一输出后端	外部 RobotHW 不能同时独占 SDK

可展示的工程贡献

感知侧：围绕旋转目标检测构建统一 ROS 消息，连接 RGB-D 时间配对、相机标定和抓取几何，把像素中心与方向转换成基座系抓取参数。对深度异常、无效方向和未标定宽度设置明确的拒绝条件。

驱动侧：把厂商 SDK 封装为 ros_control 标准硬件接口，贯通 MoveIt、关节轨迹控制器和真实控制柜；通过独立状态读取与供点线程，降低计算与网络通信之间的相互阻塞。

控制侧：实现统一 PBVS 控制核心，使用雅可比阻尼伪逆、连续软死区、关约束和有界位置队列；通过 Gazebo 与 SDK 两个输出后端复用算法，并加入目标超时与对齐保持状态。

展示时应保留的限制

ROS 定时器运行在普通调度环境中，100 Hz 或 250 Hz 属于目标节拍，不是硬实时保证。局部伺服没有直接复用 MoveIt 规划场景的碰撞检查，伺服工作区域及接近距离需先完成验证。

Gazebo 中使用的抓取附着插件是物理仿真辅助，不是实体夹爪的抓取检测。仿真成功率与实机成功率必须分别统计。仓库中的眼在手上、眼在手外以及 table、depth 模式也应按实际演示条件选择，避免展示图和启动参数不一致。

八 实验口径与招聘展示讲述

指标如何形成可复核的结果

指标	展示数值	随实验记录保存的内容
mAP@0.5	77.2%	数据集划分、类别、标注方式、模型版本与逐类 AP
位置与姿态误差	5.8 mm / 3.3°	参考真值、样本量、误差定义、均值或 RMSE；姿态角含义
模型预测耗时	平均 5.4 ms	CPU/GPU、输入尺寸、预热、样本数、是否含前后处理
单次运动规划	<100 ms	规划场景、起终点、规划器、失败次数及耗时分布
综合抓取成功率	92.6%	成功次数/总次数、物体组成、成功判据及实机/仿真条件

这些结果描述项目测试表现。招聘展示中应同时提供对应实验配置与原始记录，不将模型推理时间扩展为端到端延迟，也不将伺服死区或速度上限表述为实测精度。

建议的演示顺序

先用真实检测画面解释 OBB 为何适合任意平面摆放方向，再在 RViz 展示基座系抓取位姿、碰撞场景和规划轨迹。随后展示独立视觉伺服对目标位置变化的在线跟踪与目标丢失保持。混合模式可继续展示远距 APPROACH 到近距 TRACKING 的状态切换，结合运行日志说明阈值与目标丢失行为。

招聘交流用项目介绍

我们开发了 AUBO i5 智能分拣与视觉伺服系统，面向目标位置和摆放角度不确定的桌面分拣任务。系统将 YOLO-OBB 旋转检测与 RealSense RGB-D 数据结合，通过标定和坐标变换生成抓取位置与夹爪方向；使用 MoveIt 组织抓取、搬运和放置，并基于厂商 SDK 完成 ros_control 硬件接口和轨迹执行链路。

在视觉闭环部分，我们根据目标与 TCP 的位姿误差，通过雅可比阻尼伪逆求关节速度，完成限速、限加速度与位置积分，再利用有界队列向执行后端供点。控制计算默认 100 Hz，位置点按 200 Hz 生成，真机关节反馈 50 Hz。混合模式通过 MoveIt 规划服务与同一执行队列，衔接长距离规划和近距离视觉闭环。

项目实验中，检测 mAP@0.5 为 77.2%，位置和姿态估计误差为 5.8 mm 和 3.3°，平均模型预测耗时 5.4 ms，单次规划耗时小于 100 ms，综合抓取成功率 92.6%。讲述时结合具体测试设备和样本口径，重点解释从检测结果到稳定执行的工程链路。

九 源码索引与参数复核入口

以下路径均相对于仓库根目录。正文中的 S 编号对应这些实现入口；频率与参数以当前启动时的实际覆盖值为准。

编号	源码或配置	核对内容
S1	aubo/aubo_perception/scripts/ultralytics_yolo_node.py aubo/aubo_perception/config/ultralytics_yolo.yaml	OBB 输出、原时间戳、非阻塞推理锁、10 Hz 限频
S2	aubo/aubo_perception/scripts/yolo_rgbd_target_node.py aubo/aubo_perception/src/aubo_perception/grasp_geometry.py aubo/aubo_perception/config/grasp_pipeline.yaml	深度缓存与匹配、反投影、顶面与中心、旋转框方向
S3	aubo/aubo_sorting_core/src/color_sorting_task.cpp aubo/aubo_sorting_core/src/target_tracking.cpp aubo/aubo_sorting_core/src/pick_place.cpp	任务工作线程、目标缓存、抓放顺序、10 Hz 等待
S4	aubo/aubo_sorting_core/src/motion_executor.cpp aubo/aubo_moveit_config/config/ompl_planning.yaml aubo/aubo_sorting/config/yolo_sorting.yaml	RRTConnect、笛卡尔路径、时间参数化、规划预算
S5	aubo/aubo_ros_control/src/aubo_hw_main.cpp aubo/aubo_ros_control/src/aubo_hardware_interface.cpp aubo/aubo_ros_control/include/aubo_ros_control/aubo_hardware_interface.h	250 Hz 主循环、50 Hz 状态读取、SDK 供点、5 ms 限幅常量
S6	aubo/aubo_ros_control/src/visual_servo.cpp aubo/aubo_ros_control/src/visual_servo_common.cpp aubo/aubo_ros_control/src/direct_sdk_backend.cpp	PBVS、状态机、子步积分、队列、独立 SDK 输出线程
S7	aubo/aubo_ros_control/config/visual_servo_common.yaml aubo/aubo_ros_control/config/visual_servo_eye_to_hand.yaml aubo/aubo_ros_control/config/controllers.yaml	100/200 Hz、死区、超时、姿态目标、状态发布频率
S8	realsense-ros-development/realsense2_camera/launch/rs_camera.launch aubo/aubo_perception/models/workspace_camera/model.sdf	真实相机默认 30 Hz、固定仿真相机 20 Hz
S9	aubo/aubo_ros_control/src/hybrid_control.cpp aubo/aubo_ros_control/config/hybrid_control.yaml	规划服务、轨迹校验、距离迟滞、短队列与状态切换

相机与任务启动参数

眼在手外应显式选择 `camera_mount:=eye_to_hand`，并核对相机命名空间及标定 TF；使用深度估计高度时选择 `height_mode:=depth`。YOLO 分拣便捷入口默认采用 `eye_in_hand` 和 `table`，不能直接当作眼在手外 RGB-D 实验配置。

开源仓库 <https://github.com/2799063570/ros-mobile-manipulation-lab>